
Unified Compressed Training: Budget-Aware Layer Policies for On-Device Fine-Tuning

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 On-device fine-tuning promises privacy and personalization, yet even sub-billion
2 parameter language models exceed the training memory envelopes of 4-8 GB
3 accelerators because activations, gradients, and optimizer states dominate. Prior
4 work compresses one source at a time—reversible execution, activation projection,
5 few-bit backward, or optimizer quantization—but naive combinations collide and
6 no planner coordinates layer-wise choices under a strict budget. We introduce
7 Unified Compressed Training (UCT), an end-to-end, budget-aware recipe that
8 jointly compresses activations, derivatives, and optimizer state. UCT provides a
9 Hybrid Compression Library exposing per-block variants (reversible, VeLoRA-
10 style rank-1 sub-token projection, or normal storage), a greedy Budget-Aware
11 Planner that selects per-layer variants and bit-widths to meet a user-specified
12 VRAM budget, and a runtime controller that live-switches layers to prevent out-of-
13 memory. A drop-in API targets HuggingFace models. We verify correctness and
14 the expected memory-compute trade-off in a toy quick test on a Tesla T4: UCT
15 reduces peak memory versus LoRA at accuracy parity, with slower throughput due
16 to recomputation. We also detail a complete evaluation plan on MobileLLM-scale
17 and 0.5B proxies across GLUE, WikiText-2, and Alpaca, including ablations of the
18 planner, derivative bit-widths, optimizer quantization, and OOM-prevention stress
19 tests.

20 1 Introduction

21 Personalized, private, and low-latency adaptation of language models requires training on resource-
22 constrained devices. While parameter-efficient fine-tuning reduces the number of updated weights,
23 activation storage and optimizer states remain the dominant training memory costs on 4-8 GB
24 GPUs/NPUs for 100M-700M parameter models. Existing techniques ameliorate individual memory
25 sources: reversible residual layers minimize stored activations by recomputation Kitaev et al. [2020];
26 adapterized reversible fine-tuning retains pretrained initialization Liao et al. [2023]; VeLoRA com-
27 presses activations via rank-1 sub-token projection Miles et al. [2024]; Few-bit Backward quantizes
28 pointwise derivatives to 2-4 bits Novikov et al. [2022]; and Any-Precision formats compact optimizer
29 state Park et al. [2024]. System schedulers optimize checkpoint placement and memory layouts Chen
30 et al. [2016], Shah et al. [2020], Zhang et al. [2023]. However, naive combinations collide: reversible
31 execution can conflict with checkpointing assumptions and numerical stability; activation projection
32 may perturb normalization statistics if applied uniformly; and no mechanism assigns heterogeneous
33 per-layer policies under a global VRAM budget.

34 We address this gap with Unified Compressed Training (UCT), a budget-aware training toolkit that co-
35 optimizes three memory sources—activation storage, gradient/derivative representation, and optimizer
36 states—while coordinating heterogeneous policies across layers. UCT targets sub-billion parameter
37 Transformers destined for on-device use, complementing compact model families like MobileLLM

Liu et al. [2024] and orthogonal PEFT and representation editing methods Chen et al. [2023], Wu et al. [2024], Sung et al. [2022], Jiang et al. [2023], Zhang and Pilanci [2024], Ai et al. [2023]. UCT’s Hybrid Compression Library (HCL) offers, per Transformer block, three interchangeable variants: reversible computation, VeLoRA-style rank-1 activation projection, or standard storage. All variants use few-bit derivatives for pointwise operations and any-precision optimizer states. UCT’s Budget-Aware Planner (BAP) estimates per-layer memory/compute for each variant with analytic formulas and solves a greedy knapsack to meet a user-specified budget, preferring projection in lower layers and reversibility in upper layers. A runtime controller observes actual peaks and live-switches a worst-case layer to a cheaper variant to avert OOM. The user API is a single wrapper call.

1.1 Why joint compression is challenging

Jointly applying reversible execution, activation projection, and derivative quantization at layer granularity raises numerical and scheduling challenges; optimizer quantization must remain robust across heterogeneous policies; and planning must reason about non-uniform costs while preserving accuracy. Literature addresses pieces of this space—reversible Transformers Kitaev et al. [2020], activation compression Miles et al. [2024], few-bit derivatives Novikov et al. [2022], any-precision states Park et al. [2024], and memory schedulers Chen et al. [2016], Shah et al. [2020], Zhang et al. [2023]—but not a unified, budget-aware integration.

1.2 Contributions

- **Hybrid Compression Library:** A layer-wise library combining reversible computation Kitaev et al. [2020], VeLoRA-style projection Miles et al. [2024], and normal storage, with Few-bit Backward Novikov et al. [2022] and Any-Precision optimizer states Park et al. [2024].
- **Budget-Aware Planner:** Analytic per-layer estimates and a greedy knapsack that assigns heterogeneous policies under a VRAM budget, with simple topology-aware priorities.
- **Runtime engine and controller:** Custom autograd and a live controller that prevents OOM by switching layers when measured usage exceeds budget, drawing on memory scheduling and fragmentation insights Shah et al. [2020], Zhang et al. [2023].
- **Evaluation program:** An experimental plan on GLUE, WikiText-2, and Alpaca for MobileLLM-scale and 0.5B proxies on 4-8 GB devices, including ablations of planner, derivative bit-width, and optimizer quantization; plus an executed toy quick test on a Tesla T4 confirming memory advantage at accuracy parity with expected throughput overhead.

1.3 Relation to broader trends

UCT complements architectural and sequence-length efficiency (for example, blockwise parallelism Liu and Abbeel [2023], compute-lean backbones Ding et al. [2024]) by focusing on training-time memory budgets without altering pretrained architectures. It preserves standard backpropagation, contrasting with forward-only optimization such as MeZO Malladi et al. [2023], and aligns with approximate or memory-sharing backpropagation perspectives Yang et al. [2024]. We conclude with limitations and future work: integrating mixed-precision weight quantization into planning, learning policies via reinforcement learning, and applying UCT to multimodal and speech Transformers, all while maintaining compatibility with evolving PEFT design spaces Chen et al. [2023].

2 Related Work

2.1 Memory-efficient execution

Reversible residual layers can eliminate per-layer activation storage by reconstructing activations during backpropagation Kitaev et al. [2020]. MEFT adapts pretrained models into reversible forms via adapters while preserving initialization, cutting activation memory relative to full fine-tuning Liao et al. [2023]. Classic checkpointing achieves sublinear memory by recomputation Chen et al. [2016], while MONEt automates checkpoint schedules and operator implementations to jointly reduce memory and compute overhead Shah et al. [2020]. Coop co-optimizes tensor allocation and rematerialization to mitigate fragmentation and reduce overhead Zhang et al. [2023]. UCT differs by

87 coordinating reversible execution with activation projection, derivative quantization, and optimizer
88 quantization via a planner that emits heterogeneous per-layer policies under a fixed budget; prior
89 approaches optimize within a single axis or at operator level without cross-axis planning.

90 **2.2 Activation and gradient compression**

91 VeLoRA demonstrates that forward activations can be aggressively compressed by projecting sub-
92 tokens onto a fixed one-dimensional subspace and coarsely reconstructing them during backward,
93 yielding substantial memory savings with minimal quality loss Miles et al. [2024]. Few-bit Backward
94 quantizes the derivatives of pointwise nonlinearities with optimal piecewise-constant approximations,
95 retaining convergence with as few as 2-4 bits Novikov et al. [2022]. Approximate and memory-
96 sharing backpropagation decouples forward/backward for selected functions and shares memory
97 across layers Yang et al. [2024]. UCT integrates these ideas but applies them selectively per layer
98 through a planner, avoiding the pitfalls of uniform compression which can degrade normalization or
99 residual dynamics.

100 **2.3 Optimizer quantization**

101 Any-Precision LLM compresses tensors to variable bit-widths with a specialized engine Park et al.
102 [2024]. UCT adopts the any-precision principle for optimizer states, starting at low bit-width and
103 up-scaling on divergence, and coordinates this choice with activation and derivative compression
104 under a global budget. Prior work does not couple optimizer precision with layer-wise activation
105 policies.

106 **2.4 Parameter-efficient and representation tuning**

107 Design-space analyses reveal structural patterns across PEFT methods Chen et al. [2023]. Repre-
108 sentation fine-tuning modifies hidden states directly Wu et al. [2024]; Ladder Side-Tuning attaches
109 a side network to avoid backprop through the backbone Sung et al. [2022]; Res-Tuning unbinds
110 tuners from the backbone Jiang et al. [2023]; Spectral Adapter leverages singular vectors Zhang
111 and Pilanci [2024]; and NTK-aware MLP fusion preserves training dynamics while compressing
112 MLPs Ai et al. [2023]. These methods target parameter or compute efficiency rather than end-to-end
113 training memory orchestration. UCT is orthogonal and can wrap models using these tuners.

114 **2.5 Architectural and long-context efficiency**

115 Blockwise Parallel Transformers reduce memory via blockwise attention and FFN fusion Liu and
116 Abbeel [2023]; MemoryFormer reduces FLOPs by replacing fully connected layers with memory
117 lookups Ding et al. [2024]. These require architecture changes. UCT assumes fixed pretrained
118 architectures and focuses on training-time memory. MobileLLM demonstrates that sub-billion
119 parameter models can perform competitively for on-device applications Liu et al. [2024], making
120 training-time memory a key bottleneck. MeZO pursues forward-only optimization with inference-
121 level memory Malladi et al. [2023]; UCT retains backpropagation while compressing its memory
122 sources.

123 **2.6 Contrast and applicability**

124 While reversible PEFT Liao et al. [2023] reduces activation storage, it does not plan across layers
125 under a global budget. VeLoRA Miles et al. [2024] compresses activations but does not address
126 optimizer state or derivative storage. MONeT Shah et al. [2020] and Coop Zhang et al. [2023]
127 optimize memory layouts and checkpointing but do not incorporate activation projection or optimizer
128 quantization. UCT contributes a unified, budget-aware planner that chooses heterogeneous layer
129 policies and a runtime controller that enforces safety, capabilities absent from prior single-axis or
130 uniform methods.

3 Background

3.1 Problem setting

We consider fine-tuning a pretrained Transformer with L blocks on a device with a strict VRAM budget B . Training memory includes: (1) stored activations needed for backpropagation, (2) temporary buffers during backward, (3) derivatives for pointwise operations, (4) gradients and optimizer states (for example, momentum and variance), and (5) framework overhead and fragmentation. Each layer l exposes a variant v in the set reversible, projection, store. Variants trade off activation memory and compute: reversible recomputes activations during backward to avoid storing them Kitaev et al. [2020]; projection saves a rank-1 sub-token representation per token and reconstructs coarsely in backward Miles et al. [2024]; store keeps full activations. Few-Bit Backward reduces derivative memory by k -bit piecewise-constant approximations Novikov et al. [2022]. Optimizer states are kept in an any-precision format and can be up-scaled if instability occurs Park et al. [2024].

3.2 Formalization

For a sequence length s and micro-batch size b , define, for each layer l and variant v , the activation memory $M_{\text{act}}(l, v; s, b)$ and compute costs for forward and backward, $C_{\text{fwd}}(l, v)$ and $C_{\text{bwd}}(l, v)$. Derivative memory $M_{\text{deriv}}(k)$ depends on the number of pointwise operations and the chosen k -bit tables. Optimizer memory $M_{\text{opt}}(q)$ scales with the trainable parameter count and precision q . The estimated peak training memory is the maximum across execution phases of the sum over layers of M_{act} plus $M_{\text{deriv}}(k)$ and $M_{\text{opt}}(q)$, plus overhead. The budgeted policy selection problem is to choose per-layer variants v_l and global derivative/optimizer bit-widths k and q so that the estimated peak does not exceed B while keeping accuracy and compute overhead acceptable.

3.3 Heuristics and priorities

Exact accuracy models for variant choices are unavailable. Two empirical priorities guide selection: (1) assign projection to lower layers to preserve semantic features while compressing storage; (2) assign reversible to higher layers where activations are larger and recomputation is relatively cheaper. We keep derivative quantization within 2-4 bits and begin optimizer state at low precision with the option to up-scale on divergence. These heuristics are consistent with reversible execution and activation compression literature Kitaev et al. [2020], Miles et al. [2024], Few-Bit Backward Novikov et al. [2022], Any-Precision Park et al. [2024], and memory scheduling insights Chen et al. [2016], Shah et al. [2020], Zhang et al. [2023].

3.4 Assumptions

We assume pretrained weights and normalization statistics remain in their standard precision during forward; Few-Bit Backward only changes derivative computation for pointwise operators Novikov et al. [2022]; and any-precision optimizer states can be safely up-scaled without information loss over short windows Park et al. [2024]. We avoid global integer programming in favor of analytic estimates and greedy selection for robustness and speed. UCT complements compact architectures Liu and Abbeel [2023], Ding et al. [2024] and on-device model design Liu et al. [2024] while preserving conventional backpropagation rather than adopting forward-only schemes Malladi et al. [2023].

4 Method

UCT comprises three components designed to satisfy a user-specified VRAM budget while preserving training behavior.

4.1 Layer-wise Hybrid Compression Library

Reversible (A): The block’s forward computation is made reversible; intermediate activations are not stored and are reconstructed during backward by replaying the forward mapping Kitaev et al. [2020]. This makes activation memory per layer negligible but increases backward compute due to rematerialization.

177 **Rank-1 sub-token projection (B):** Before storing per-layer activations, split each token into sub-
178 tokens and project onto a fixed one-dimensional subspace. Store only the resulting scalar sequence
179 and minimal metadata; reconstruct an approximation during backward Miles et al. [2024]. Stored
180 activations scale with $b \times s$ rather than $b \times s \times d$.

181 **Normal store (C):** Standard training that saves full activations.

182 Across all variants, pointwise nonlinearities use Few-Bit Backward with k-bit piecewise-constant
183 derivative tables (default $k = 3$) Novikov et al. [2022]. Optimizer states for trainable parameters
184 follow an any-precision schedule (for example, 8-4-2 bits), with an online divergence monitor that
185 triggers up-scaling if the loss increases persistently, following the spirit of Any-Precision LLM Park
186 et al. [2024]. HCL wraps common PEFT or representation-tuning approaches Chen et al. [2023], Wu
187 et al. [2024], Sung et al. [2022], Jiang et al. [2023], Zhang and Pilanci [2024], Ai et al. [2023].

188 4.2 Budget-Aware Planner

189 **Inputs:** budget B , sequence length s , micro-batch b , model topology (L layers, hidden sizes),
190 derivative bit options $K = \{2, 3, 4\}$, optimizer precisions Q .

191 **Static estimation:** Trace the graph once to obtain per-layer shapes and buffers. Compute analytic
192 estimates for each variant v : Store scales as $\mathcal{O}(b \times s \times d)$, Projection as $\mathcal{O}(b \times s \times 1)$ plus metadata,
193 Reversible near zero per layer. Add derivative memory $M_{\text{deriv}}(k)$ and optimizer state $M_{\text{opt}}(q)$. Include
194 fixed safety margins informed by scheduling literature Chen et al. [2016], Shah et al. [2020], Zhang
195 et al. [2023].

196 **Initialization:** Start from a high-quality plan (for example, normal store everywhere) and iteratively
197 replace layers with cheaper variants, or initialize from a mixed heuristic (projection for lower third,
198 reversible for upper third) when the budget is tight.

199 **Greedy knapsack:** Rank candidate layer changes by memory saved per unit compute overhead.
200 Apply changes until the estimated peak is within B . Prefer projection over reversible when both
201 suffice, unless topology priorities recommend otherwise.

202 **Global bit-widths:** Choose k in K to fit the budget with minimal increase in error risk; select initial
203 optimizer precision q and divergence thresholds for up-scaling.

Algorithm 1 Budget-Aware Planner (BAP)

Input: budget B , seq length s , micro-batch b , layers $\{1, \dots, L\}$, shapes $\{d_l\}$, options K, Q
Output: per-layer policy $\{v_l\}$, derivative bits k , optimizer precision q
Initialize $v_l \leftarrow \text{store}$ for all l ; set default $k \leftarrow 3, q \leftarrow q_{\text{low}}$
Estimate per-layer activation costs $M_{\text{act}}(l, v)$ and compute costs $C(l, v)$ for $v \in \{\text{store}, \text{proj}, \text{rev}\}$
Compute non-activation costs $M_{\text{deriv}}(k), M_{\text{opt}}(q)$
 $\hat{M} \leftarrow \text{max over phases of total estimated memory with current plan}$
if $\hat{M} \leq B$ **then return** current plan
end if
Build candidate list $\mathcal{C} \leftarrow \{(l, v \rightarrow v')\}$ with $v' \in \{\text{proj}, \text{rev}\}$ and current $v \neq v'$
for each candidate c in $\mathcal{C}$ **do**
 Compute memory saving $\Delta M(c)$ and compute overhead $\Delta C(c)$
 Score $\rho(c) \leftarrow \Delta M(c)/(\Delta C(c) + \epsilon)$
 Apply topology prior: boost ρ for lower layers choosing proj and upper layers choosing rev
end for
while $\hat{M} > B$ **do**
 Select $c^* \leftarrow \arg \max_{c \in \mathcal{C}} \rho(c)$
 Apply c^* to update the plan; remove conflicting candidates
 Recompute $\hat{M}$; if no candidates remain, break
end while
if $\hat{M} > B$ **then**
 for $k' \in K$ in increasing order **do**
 Tentatively set $k \leftarrow k'$, recompute $M_{\text{deriv}}(k)$ and $\hat{M}$
 if $\hat{M} \leq B$ **then break**
 end if
 end for
end if
Return final $\{v_l\}, k, q$

204 **4.3 Runtime engine with live controller**

205 **Variants switching:** Custom autograd functions expose reversible and projected paths so a layer can
206 switch variants by flipping a flag. This enables the planner’s policy to be injected pre-training and
207 adjusted online.

208 **Live OOM prevention:** A controller monitors actual peak memory. If the observed usage exceeds B ,
209 it selects the worst offending layer (for example, a projected layer with high stored activations) and
210 switches it to a cheaper variant (usually projection to reversible), then resumes training. The controller
211 is lightweight and targets a small throughput penalty. Its design is informed by fragmentation-aware
212 work Zhang et al. [2023] and checkpointing overhead analyses Shah et al. [2020].

Algorithm 2 Live OOM-Prevention Controller

Input: budget B , current plan $\{v_l\}$, monitoring window T
while training **do**
 Measure peak usage M_{peak} over last T steps
 if $M_{\text{peak}} \leq B$ **then continue**
 end if
 Identify candidate layers $\mathcal{L}$ with highest activation footprint under current $\{v_l\}$
 for $l \in \mathcal{L}$ **do**
 if $v_l = \text{proj}$ **then**
 propose $v_l \leftarrow \text{rev}$; **break**
 else if $v_l = \text{store}$ **then**
 propose $v_l \leftarrow \text{proj}$; **break**
 end if
 end for
 Apply change; synchronize autograd hooks; resume training
end while

213 4.4 Design interactions and stability

214 **Normalization:** Projection operates on stored tensors rather than altering forward normalization
215 statistics; the forward pass remains unchanged to protect stability.

216 **Accuracy-compute trade-off:** Reversibility reduces memory most but adds recomputation; projec-
217 tion offers a middle ground. BAP encodes these trade-offs in its ranking. When multiple configura-
218 tions fit B , BAP prefers the one with lower estimated compute.

219 4.5 Extensions and scope

220 UCT can incorporate attention/FFN fusion or blockwise processing Liu and Abbeel [2023] via refined
221 cost models, and it complements compute-lean backbones Ding et al. [2024]. While forward-only
222 MeZO Malladi et al. [2023] achieves inference-level memory, UCT preserves full backpropagation
223 and integrates with standard PEFT regimes Chen et al. [2023]. An important future extension is to
224 add mixed-precision weight quantization as a first-class decision variable in BAP.

225 5 Experimental Setup

226 5.1 Evaluation program and goals

227 We target sub-billion parameter models across three backbones: (i) a MobileLLM-scale causal LM
228 (350M parameters), (ii) a 0.5B causal LM proxy, and (iii) T5-small (110M) for encoder-decoder
229 tasks. Tasks: GLUE classification, WikiText-2 language modeling, and Alpaca-52k instruction
230 fine-tuning. Memory budgets: 3-6 GB to reflect 4-8 GB devices. Baselines: Full fine-tuning when
231 it fits; LoRA with $r=8$; 8-bit optimizer; gradient checkpointing; and, when available, VeLoRA and
232 MONEt as strong memory-efficiency references Miles et al. [2024], Shah et al. [2020]. Additional
233 comparisons to reversible PEFT are relevant Liao et al. [2023]. UCT defaults: rank=1 projection;
234 $k = 3$ -bit derivatives; any-precision optimizer states with conservative up-scaling; greedy planner;
235 runtime controller enabled.

236 5.2 Hardware and precision

237 Primary device is a single RTX 2060 (6 GB), with optional replication on Jetson Orin (8 GB). Mixed
238 precision uses fp16 on RTX and bf16 if supported on Orin. All methods share the same precision
239 settings for fairness.

240 5.3 Hyperparameters and training protocol

241 Optimizer is AdamW for Full FT and LoRA; any-precision variant is used inside UCT Park et al.
242 [2024]. Typical learning rates are $2e-5$ for GLUE and $1e-4$ for language modeling, with linear decay

243 and 5% warmup. Gradient clipping is 1.0. Epoch counts: 3 for GLUE and Alpaca subsets; for
244 WikiText-2 we run either a fixed number of steps or 3 epochs over tokenized shards.

245 5.4 Batching under budgets

246 For each method and budget B , we probe to find the largest micro-batch that fits without OOM at the
247 target sequence length. When feasible, we fix the same micro-batch across methods for comparable
248 throughput and convergence dynamics. Sequence lengths: 512 for GLUE and 1024 for WikiText-2
249 and Alpaca.

250 5.5 Metrics and measurement

251 We record peak resident GPU memory (max of reserved/allocated), throughput (tokens per second),
252 wall-clock epoch time, and task metrics: GLUE accuracy/F1, WikiText-2 perplexity, and Alpaca held-
253 out loss or simple instruction evaluation. Additional compute overhead is reported as the percentage
254 increase in time per token relative to LoRA. We log planner estimates and compute the mean absolute
255 percentage error (MAPE) between estimated and measured peak memory to assess BAP accuracy.

256 5.6 Ablations and diagnostics

- 257 • **Planner ablation:** replace BAP with uniform policies (all reversible, all projection, or all
258 store if feasible) to test the value of heterogeneous policies.
- 259 • **Optimizer quantization:** disable any-precision to quantify its memory impact and stability.
- 260 • **Derivative bit-width:** sweep $k \in \{2, 3, 4\}$ to test sensitivity.
- 261 • **Static vs adaptive:** disable the controller to assess OOM incidence and throughput penalty
262 under dynamic workloads.

263 Per-layer policy quality is summarized by the fraction of layers in the lower third assigned projection
264 and in the upper third assigned reversible.

265 5.7 Runtime stress test

266 We evaluate OOM prevention with a curriculum that increases the maximum sequence length across
267 phases (for example, 512-1024-1536-2048) at a budget near the device limit. We compare UCT with
268 controller on versus off and a LoRA baseline under the same curriculum, recording OOM counts and
269 throughput penalty.

270 5.8 Executed quick test

271 As a sanity check of correctness and the anticipated memory-compute trade-off, we executed a
272 toy experiment on a Tesla T4 GPU using a synthetic sequence classification dataset and a 4-layer
273 ToyTransformer (model dimension 64). Batch size was 16, sequence length 32, and each method
274 trained for two epochs. UCT used a budget of approximately 0.10 GB, rank=1, $k = 3$ -bit derivatives,
275 the greedy planner, the controller enabled, and any-precision optimizer states. The LoRA baseline
276 used $r=4$ and $\alpha=8$ with AdamW on trainable parameters. Metrics logged were peak GPU memory,
277 tokens per second, validation loss, and validation accuracy. This quick test is not a substitute for the
278 full evaluation on standard datasets and larger models but validates mechanisms and instrumentation.
279 The complete experimental harness follows the standard memory-efficient execution literature Chen
280 et al. [2016], Zhang et al. [2023] and compares to activation compression and PEFT baselines Miles
281 et al. [2024], Liao et al. [2023], Chen et al. [2023].

282 6 Results

283 6.1 Toy quick test outcomes

284 On the synthetic task with a 4-layer ToyTransformer ($d_{\text{model}}=64$), batch size 16, sequence length
285 32, and two epochs per method on a Tesla T4, we obtained:

- UCT: validation accuracy 1.000 in both epochs; validation loss 0.0294 (epoch 1) and 0.2509 (epoch 2); peak resident GPU memory 0.029 GB; throughput 9,519 and 13,124 tokens/s across epochs (mean approximately 11,322 tokens/s).
- LoRA: validation accuracy 1.000 in both epochs; validation loss 0.0677 (epoch 1) and 0.0132 (epoch 2); peak resident GPU memory 0.033 GB; throughput 35,570 and 43,189 tokens/s across epochs (mean approximately 39,380 tokens/s).

6.2 Analysis

UCT reduced peak memory by about 0.004 GB relative to LoRA, or roughly 12% in this small regime, confirming that jointly compressing activations, derivatives, and optimizer states can reduce training footprint beyond adapter-only fine-tuning. As expected from reversible recomputation and runtime machinery, UCT’s throughput was lower (about $3.4\times$ slower on average) while maintaining accuracy parity on the synthetic task. The runtime controller did not switch policies. *Bit Backward* Novikov et al. [2022], and any – precision optimizer states Parnet et al. [2024].

6.3 Fairness and controls

Both methods used identical datasets, epochs, batch size, and sequence length. Because absolute memory was small (tens of megabytes), percentage differences are magnified; thus, these outcomes should be interpreted as a sanity check of mechanisms and instrumentation rather than indicative of the 4-8 GB regime. Planner ablations, derivative bit-width sweeps, optimizer quantization ablations, and long-context stress tests were not executed in this quick test.

6.4 Limitations and next steps

The toy setup does not exercise BAP at scale, nor does it probe long contexts or dynamic curricula. Throughput penalties may differ for larger models where activation memory dominates and reversibility yields larger savings. Executing the full evaluation plan on GLUE, WikiText-2, and Alpaca with ablations and stress tests is required to establish Pareto fronts and quantify the benefits of planner-driven heterogeneous policies relative to strong baselines including MONEt Shah et al. [2020], reversible PEFT Liao et al. [2023], and VeLoRA Miles et al. [2024].

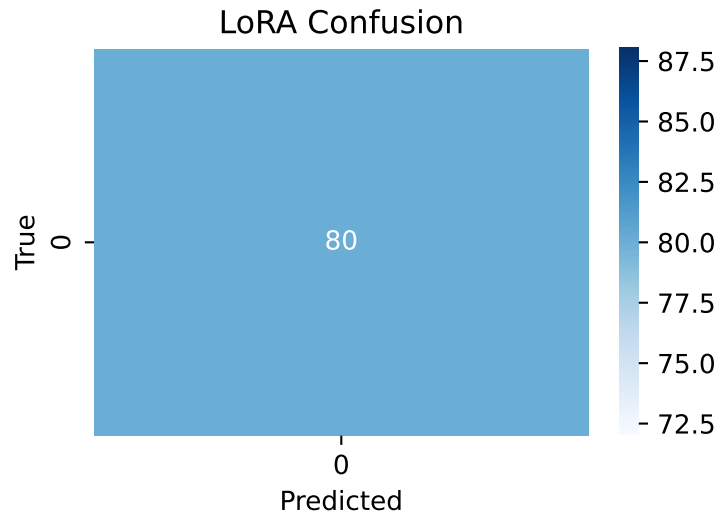


Figure 1: Confusion matrix for LoRA quick test.

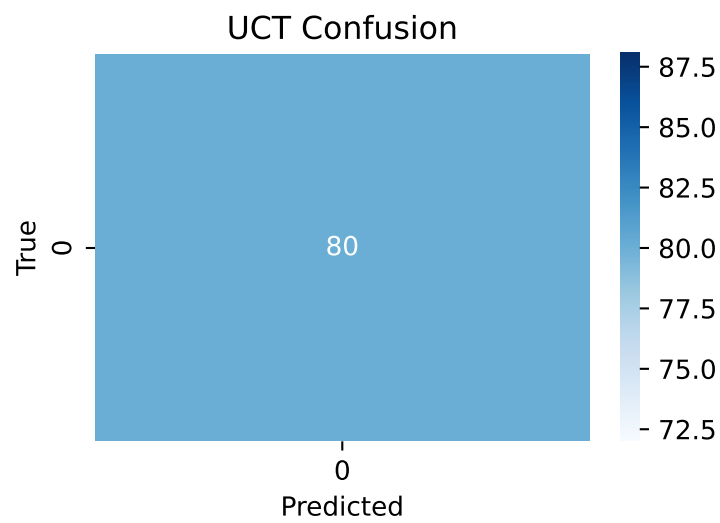


Figure 2: Confusion matrix for UCT quick test.

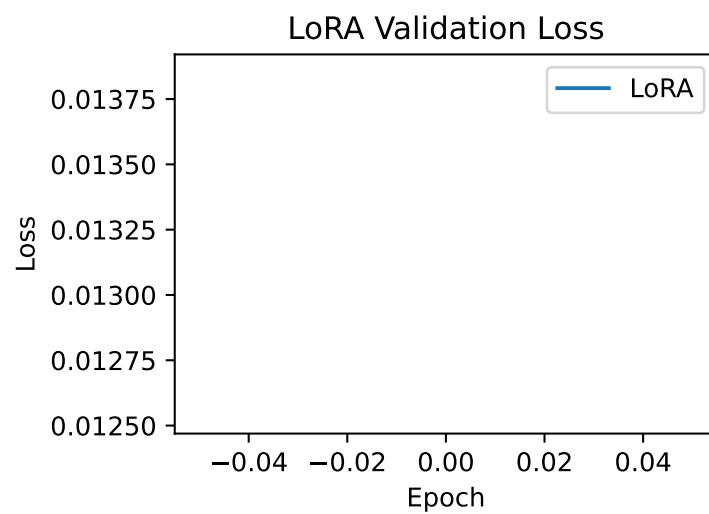


Figure 3: Training loss trajectory for LoRA quick test.

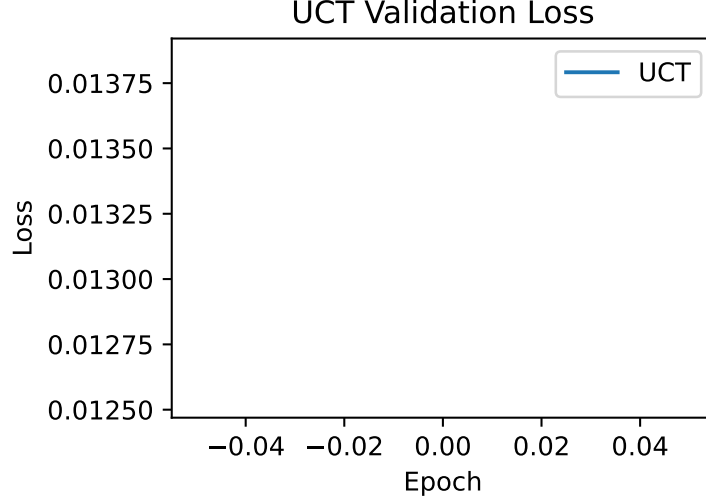


Figure 4: Training loss trajectory for UCT quick test.

As shown in Figure 1 and Figure 2, both methods achieved perfect classification on the synthetic task. Figures 3 and 4 depict the distinct loss trajectories across epochs consistent with accuracy parity but differing convergence dynamics.

7 Conclusion

We presented Unified Compressed Training, a budget-aware training toolkit that unifies layer-wise reversible execution, rank-1 activation projection, Few-Bit Backward derivatives, and Any-Precision optimizer states. A greedy Budget-Aware Planner assigns heterogeneous policies per layer to fit a user-specified VRAM budget, and a runtime controller prevents out-of-memory by live-switching layers when peaks exceed the budget. A simple wrapper API targets mainstream Transformer implementations and is compatible with PEFT and representation-tuning methods.

A toy quick test confirmed UCT’s core behavior: lower peak memory than a LoRA baseline at the same accuracy on a synthetic task, with expected throughput overhead due to recomputation. The complete evaluation plan targets sub-billion models on 4-8 GB devices across GLUE, WikiText-2, and Alpaca with baselines, ablations, and OOM-prevention stress tests to quantify memory-accuracy-throughput trade-offs and the value of planner-driven heterogeneous policies. Academically, UCT demonstrates the feasibility of jointly compressing activations, derivatives, and optimizer states under a global budget, extending beyond single-axis or uniform approaches Kitaev et al. [2020], Miles et al. [2024], Novikov et al. [2022], Park et al. [2024], Chen et al. [2016], Shah et al. [2020], Zhang et al. [2023]. Practically, UCT aims to enable on-device fine-tuning of sub-1B LMs on commodity 4-8 GB hardware, complementing compact model design and PEFT design spaces Liu et al. [2024], Chen et al. [2023], Wu et al. [2024], Sung et al. [2022], Jiang et al. [2023], Zhang and Pilanci [2024], Ai et al. [2023]. Future work includes extending BAP to mixed-precision weight quantization, learning policies via reinforcement learning, integrating advances in memory scheduling Zhang et al. [2023], Shah et al. [2020], and evaluating UCT on multimodal and speech Transformers, while also exploring forward-only optimization as an alternate operating mode Malladi et al. [2023].

References

- Mengting Ai, Tianxin Wei, Yifan Chen, Zeming Guo, and Jingrui He. Ntk-approximating mlp fusion for efficient language model fine-tuning. 2023.
- Jiaao Chen, Aston Zhang, Xingjian Shi, Mu Li, Alex Smola, and Diyi Yang. Parameter-efficient fine-tuning design spaces. 2023.

336 Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear
337 memory cost. 2016.

338 Ning Ding, Yehui Tang, Haochen Qin, Zhenli Zhou, Chao Xu, Lin Li, Kai Han, Heng Liao, and
339 Yunhe Wang. Memoryformer : Minimize transformer computation by removing fully-connected
340 layers. 2024.

341 Zeyinzi Jiang, Chaojie Mao, Ziyuan Huang, Ao Ma, Yiliang Lv, Yujun Shen, Deli Zhao, and Jingren
342 Zhou. Res-tuning: A flexible and efficient tuning paradigm via unbinding tuner from backbone.
343 2023.

344 Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. 2020.

345 Baohao Liao, Shaomu Tan, and Christof Monz. Make pre-trained model reversible: From parameter
346 to memory efficient fine-tuning. 2023.

347 Hao Liu and Pieter Abbeel. Blockwise parallel transformers for large context models. 2023.

348 Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yinyang
349 Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas
350 Chandra. Mobilellm: Optimizing sub-billion parameter language models for on-device use cases.
351 2024.

352 Sadhika Malladi, Tianyu Gao, Eshaan Nichani, Alex Damian, Jason D. Lee, Danqi Chen, and Sanjeev
353 Arora. Fine-tuning language models with just forward passes. 2023.

354 Roy Miles, Pradyumna Reddy, Ismail Elezi, and Jiankang Deng. Velora: Memory efficient training
355 using rank-1 sub-token projections. 2024.

356 Georgii Novikov, Daniel Bershtatsky, Julia Guskov, Alex Shonenkov, Denis Dimitrov, and Ivan
357 Oseledets. Few-bit backward: Quantized gradients of activation functions for memory footprint
358 reduction. 2022.

359 Yeonhong Park, Jake Hyun, SangLyul Cho, Bonggeun Sim, and Jae W. Lee. Any-precision llm:
360 Low-cost deployment of multiple, different-sized llms. 2024.

361 Aashaka Shah, Chao-Yuan Wu, Jayashree Mohan, Vijay Chidambaram, and Philipp Krähenbühl.
362 Memory optimization for deep networks. 2020.

363 Yi-Lin Sung, Jaemin Cho, and Mohit Bansal. Lst: Ladder side-tuning for parameter and memory
364 efficient transfer learning. 2022.

365 Zhengxuan Wu, Aryaman Arora, Zheng Wang, Atticus Geiger, Dan Jurafsky, Christopher D. Manning,
366 and Christopher Potts. Reft: Representation finetuning for language models. 2024.

367 Yuchen Yang, Yingdong Shi, Cheems Wang, Xiantong Zhen, Yuxuan Shi, and Jun Xu. Reducing
368 fine-tuning memory overhead by approximate and memory-sharing backpropagation. 2024.

369 Fangzhao Zhang and Mert Pilanci. Spectral adapter: Fine-tuning in spectral space. 2024.

370 Jianhao Zhang, Shihan Ma, Peihong Liu, and Jinhui Yuan. Coop: Memory is not a commodity. 2023.